



EDITORA AGÊNTICA

1ª Edição · 2026

O RADAR: VERIFICAÇÃO ADVERSARIAL E EVIDÊNCIA

Uma leitura prática sobre o desenvolvimento orientado a agentes

Heverton Eduardo Peres

ENGENHEIRO DE SOFTWARE & MAKER

Heverton Eduardo Peres

O Radar: Verificação Adversarial e Evidência

Obra técnica de literatura especializada, produzida e diagramada conforme as normas ABNT para publicação editorial.

São Paulo
2026

Dados Internacionais de Catalogação na Publicação (CIP)

P667r PERES, Heverton Eduardo

O Radar: Verificação Adversarial e Evidência / Heverton Eduardo Peres. —
São Paulo : Fábrica Agêntica de Livros,
2026.

31 p. ; 21 cm.

ISBN 978-65-00000-30-6

1. Radar. 2. Verificação. 3. Adversarial. 4. Evidência. I. Título.

CDD 006.3

Sumário

O Radar: Verificação Adversarial e Evidência	6
Capítulo 6: O Radar: Verificação Adversarial e Evidência	7
Introdução	7
Explica	7
Ilustra	8
Técnica	9
A Esteira de Verificação em Três Camadas	9
O Revisor Adversarial Automatizado	10
Evidência Antes de Afirmação no CI	11
A Matriz de Casos de Teste como Artefato	12
O Modelo de Rastreabilidade de Evidência	12
O Modelo de Cobertura de Refutação	13
O Modelo de Verificação por Camadas com Evidência	14
O Padrão de Evidência em Cada Camada	14
O Padrão de Evidência em Cada Camada	15
O Modelo de Rastreabilidade Evidência-Requisito	16
A Fila de Refutação com Prioridade por Risco	16
A Fila de Refutação com Prioridade por Risco	17
O Modelo de Calibragem do Revisor Automatizado	18
A Amostragem de Pareceres como Auditoria	19
O Modelo de Cobertura de Refutação por Risco	19
O Radar do Radar	20
O Orçamento de Verificação por Camada	20
O Modelo de Gate de Refutação por Critério	21
A Curva de Custo da Refutação	21
O Painel de Saúde do Radar	21
Passos para Implantar o Radar	22
Aplica	22
Conclusão	23
Por que este capítulo importa	23
Conceitos-chave deste capítulo	23
Checklist para aplicar hoje	24
Perguntas que você deve se fazer	24
Glossário rápido	24

O erro mais comum nesta fase	25
Um exemplo concreto para fixar	25
A rotina de quem já opera assim	25
O que não fazer: os anti-padrões mais comuns	26
Como medir o progresso na prática	26
O papel do líder neste capítulo	26
Perguntas frequentes honestas	27
Um convite para a prática deliberada	27
Síntese para levar com você	27
De onde veio a necessidade desta mudança	28
Conversando com quem resiste	28
O dia a dia no detalhe	28
O custo invisível que decide tudo	29
Uma visão de longo prazo	29
Próximos Passos	30

O Radar: Verificação Adversarial e Evidência

Uma leitura direta e prática para quem quer levar o desenvolvimento orientado a agentes a sério — sem jargão acadêmico, com exemplos aplicáveis.

Capítulo 6: O Radar: Verificação Adversarial e Evidência

Introdução

No Capítulo 5, você ligou os motores — harness, skills, MCPs e worktrees — e adotou o test-first como régua do build. Agora chegou a hora do instrumento que separa o SDLC AI-first de um caos com boa intenção: o radar. A verificação adversarial é a camada que tenta refutar o trabalho antes que ele decole para produção.

Este capítulo aprofunda a Fase 5 do ciclo: as três camadas de verificação (máquina, adversarial e humana), a evidência antes de afirmação como princípio inegociável, e a implementação prática de uma esteira de refutação. Você vai aprender a construir um revisor agêntico que procura o defeito em vez de validar o acerto — e a nunca aceitar “está pronto” sem o output do comando que prova.

Explica

A verificação tradicional de software pergunta: “este código funciona?”. A verificação adversarial pergunta: “onde este código quebra?”. A diferença parece sutil, mas muda a postura de quem revisa — e o resultado da revisão.

No SDLC clássico, a verificação é uma fase final: o QA testa depois que o desenvolvimento termina, e o retrabalho volta para o desenvolvedor com um ticket. No SDLC AI-first, a verificação é uma **camada contínua** que atravessa todas as fases: o radar monitora o voo do agente do início ao fim, não apenas na aterrissagem.

A primeira camada é a máquina: typecheck, lint, testes. É a camada barata e implacável — não discute, executa. Um typecheck pega o erro que o revisor humano jamais veria em uma leitura; uma suíte de testes pega a regressão que a leitura de código não alcança. A máquina é o primeiro radar porque é o mais confiável.

A segunda camada é a adversarial: um revisor — humano ou agêntico — que assume o artefato culpado até prova em contrário. Essa postura de refutação é o antídoto para o viés

do produtor: quem escreve o código acredita que ele funciona (acabou de escrevê-lo); quem revisa precisa duvidar por profissão. A pesquisa em engenharia de software agêntica mostra que sistemas com revisão independente superam sistemas de auto-validação — o auto-testado do próprio agente tende a confirmar os próprios pressupostos.

A terceira camada é a humana: a decisão de merge. A máquina e o revisor agêntico filtram o volume; o humano arbitra a exceção. O papel do humano na verificação não é ler tudo — é decidir onde a máquina e o agente podem estar errados juntos: mudanças de contrato, impacto em produção, decisões estratégicas.

O princípio que sustenta as três camadas é a **evidência antes de afirmação**. “Está pronto” é uma afirmação; o output de um comando que passou é uma evidência. “O teste cobre o caso de borda” é afirmação; o teste que falha antes e passa depois é evidência. A esteira AI-first exige evidência em cada transição de fase — e o artefato que não tem evidência não avança.

Há uma dimensão econômica crucial: verificação antecipada é o melhor investimento de tokens do ciclo inteiro. Refutar um artefato na Fase 5 custa uma fração do que custaria corrigir o mesmo defeito em produção na Fase 7 — e a fração é medida em tokens, o recurso escasso.

Por fim, a verificação adversarial não é negatividade — é risco calculado. O revisor não diz “isso é ruim”; diz “isso falha neste cenário, e aqui está a evidência”. A refutação com evidência é o vocabulário profissional do radar: objetiva, pontual e construtiva.

Ilustra

O radar de aproximação de um aeroporto não elogia o piloto. Ele informa: altitude baixa, desvio de rota, velocidade acima do limite. Quando o piloto informa “pousando”, o radar não responde “parabéns” — responde com a confirmação objetiva: “na final, autorizado, pista livre”. O radar é o sistema que não acredita em palavras; acredita em instrumentos.

A verificação adversarial funciona exatamente assim. O agente diz “implementei a feature”. O radar responde: “rode os testes, mostre o output”. O agente diz “os testes passaram”. O radar responde: “e o caso de borda da sessão expirada? Rode também”. Evidência por evidência, o voo avança — ou volta.

[Diagrama do capítulo omitido neste formato.]

Como Comandante de Operações de Software, você vê o fluxo como uma espiral de evidência: o artefato só avança quando cada camada o libera com prova — nunca com promessa.

Técnica

A Esteira de Verificação em Três Camadas

Vamos construir a esteira de verificação como código. Cada camada produz um parecer com evidência estruturada — não opinião.

```
from dataclasses import dataclass, field
from typing import List, Optional

@dataclass
class Evidencia:
    tipo: str          # "saida_comando" | "trecho_diff" | "metrica"
    conteudo: str
    ok: bool

@dataclass
class Parecer:
    camada: str
    aprovado: bool
    evidencias: List[Evidencia] = field(default_factory=list)
    observacoes: List[str] = field(default_factory=list)

    def registrar(self, tipo: str, conteudo: str, ok: bool) -> None:
        self.evidencias.append(Evidencia(tipo, conteudo, ok))

    def parecer_final(self) -> str:
        status = "APROVADO" if self.aprovado else "REPROVADO"
        linhas = [f"[{status}] camada={self.camada}"]
        for e in self.evidencias:
            linhas.append(f"  - ({'OK' if e.ok else 'FALHA'}) {e.tipo}: {e.conteudo[:120]}")
        for o in self.observacoes:
            linhas.append(f"  ! {o}")
        return "\n".join(linhas)

    def camada_maquina(artefato: str) -> Parecer:
        parecer = Parecer("maquina", aprovado=True)
        # Simulacao: executa typecheck/lint/testes e coleta saida
        parecer.registrar("saida_comando", "typecheck: 0 erros em 142 arquivos", True)
```

```

parecer.registrar("saida_comando", "pytest: 47 passed, 0 failed", True)
parecer.registrar("saida_comando", "lint: 0 violacoes", True)
return parecer

def camada_adversarial(artefato: str) -> Parecer:
    parecer = Parecer("adversarial", aprovado=True)
    parecer.registrar("trecho_diff", "revisor independente: 3 cenarios de
borda testados", True)
    parecer.registrar("metrica", "cobertura de casos de borda: 3/3", True)
    parecer.observacoes.append(
        "recomendacao: adicionar teste de fatura duplicada em proxima
iteracao")
    return parecer

if __name__ == "__main__":
    artefato = "feature/pagamentos"
    pareceres = [camada_maquina(artefato), camada_adversarial(artefato)]
    for p in pareceres:
        print(p.parecer_final())
        print("")
    print("DECISAO HUMANA: merge autorizado" if all(p.aprovado for p in
pareceres)
        else "DECISAO HUMANA: exigir correcao")

```

A estrutura de evidência é o ponto: cada parecer registra o *tipo* de evidência, o *conteúdo* e o *status*. O revisor humano não precisa confiar na palavra do agente — lê as evidências e decide.

O Revisor Adversarial Automatizado

O revisor adversarial agêntico pode ser parametrizado para procurar classes específicas de defeito. O trecho abaixo implementa um revisor que caça “cenários de borda ausentes” em specs:

```

import re

RE_OPERADORES_BORDA = re.compile(
    r"\b(se\s+n(?:[ãa]o)|quando|apenas|somente|exceto|limite|zero|vazio|"
    r"duplicad[oa]|concorrente|timeout|expirad[oa])\b", re.IGNORECASE)

def revisar_spec(spec: str) -> list:
    """Procura declaracoes de escopo sem casos de borda associados."""
    achados = []
    for m in re.finditer(r"R\d+:\s*([^\n]+)", spec):

```

```

requisito = m.group(1)
tem_borda = bool(RE_OPERADORES_BORDA.search(requisito))
tem_teste = "criterio" in spec[m.start():m.end() + 400].lower()
if tem_borda and not tem_teste:
    achados.append(
        f"{m.group(0)[:60]} -> declara condicao de borda sem
criterio de aceite")
return achados

SPEC_SUSPEITA = """
R1: Usuario autentica com email e senha validos
R2: Quando a sessao expira, o sistema redireciona para login
R3: Apenas administradores podem excluir faturas
"""

for achado in revisar_spec(SPEC_SUSPEITA):
    print(f"[REFUTACAO] {achado}")

```

O revisor não valida — caça. Cada achado é uma refutação com evidência textual, pronta para virar ticket de correção.

Evidência Antes de Afirmação no CI

A integração contínua é a materialização do princípio: o merge só acontece se a evidência da esteira estiver verde. O script abaixo simula a porta de evidência:

```

#!/usr/bin/env bash
# Porta de evidencia: so mergeia se TODAS as camadas passarem com saida
registrada

set -euo pipefail

echo "== Camada 1: maquina =="
python -m pytest --quiet > /tmp/evidencia_teste.txt
echo "pytest ok ($(wc -l < /tmp/evidencia_teste.txt) linhas de saida)"

echo "== Camada 2: adversarial =="
python scripts/revisor-adversarial.py > /tmp/evidencia_revisao.txt
if grep -q "REFUTACAO" /tmp/evidencia_revisao.txt; then
    echo "revisor encontrou refutacoes; bloqueando merge"
    exit 1
fi
echo "revisor adversarial: nenhuma refutacao com evidencia"

echo "== Porta de evidencia liberada =="

```

O padrão é visível: cada camada grava sua saída em arquivo, e a porta de evidência exige que todas estejam limpas. A palavra “confio” não aparece em lugar nenhum — só saídas de comando.

A Matriz de Casos de Teste como Artefato

A verificação adversarial só é sistemática se os casos forem artefatos — não intuição. A matriz abaixo é o padrão: para cada requisito, os cenários feliz, borda e falha, com o teste que os protege.

```
{
  "requisito": "R3: sessao expira apos 30 minutos de inatividade",
  "cenarios": [
    {"tipo": "feliz", "descricao": "sessao ativa dentro de 30 min",
    "teste": "sessao_ativa_dentro_do_limite"},
    {"tipo": "borda", "descricao": "sessao expira exatamente em 30 min",
    "teste": "sessao_expira_no_limite_exato"},
    {"tipo": "borda", "descricao": "inatividade continua durante requisicao
longa", "teste": "requisicao_longa_renova_sessao"},
    {"tipo": "falha", "descricao": "relogio do servidor adiantado em 2
min", "teste": "sessao_com_skew_de_relogio"},
    {"tipo": "falha", "descricao": "duas sessoes concorrentes no mesmo
usuario", "teste": "sessoes_concorrentes_independentes"}
  ]
}
```

O revisor adversarial usa a matriz como régua: o agente que cobre só o cenário feliz é reprovado com evidência — faltam os cenários de borda e falha que a matriz exige.

O Modelo de Rastreabilidade de Evidência

A evidência precisa ser rastreável até a origem — o parecer que cita uma saída de comando sem apontar o arquivo é evidência órfã. O modelo abaixo é o registro de rastreabilidade:

```
{
  "evidencias": [
    {
      "id": "EVI-118",
      "camada": "maquina",
      "afirmacao": "testes passam",
      "origem": "artefatos/resultado_ci.txt",
      "linha_origem": 42,
      "verificavel": true
    },
    {
```

```

    "id": "EVI-119",
    "camada": "adversarial",
    "afirmacao": "caso de borda de sessao coberto",
    "origem": "artefatos/parecer_adversarial.md",
    "linha_origem": 15,
    "verificavel": true
  }
],
"regra": "evidencia sem origem verificavel nao conta"
}

```

A rastreabilidade de evidência é a régua final do radar: toda afirmação de verificação aponta para um artefato e uma linha — e o auditor pode conferir. A evidência órfã é rejeitada pelo mesmo princípio que rejeita citação órfã no texto.

O Modelo de Cobertura de Refutação

A eficácia do radar se mede pela cobertura de refutação — quantos cenários de borda e falha o revisor adversário testou, em relação ao que a matriz exigia. O modelo abaixo calcula a cobertura:

```

def cobertura_refutacao(matriz: dict, testados: set) -> dict:
    cenarios = {c["teste"] for c in matriz["cenarios"]}
    cobertos = cenarios & testados
    pct = round(len(cobertos) / len(cenarios) * 100, 1) if cenarios else
100.0
    faltam = sorted(cenarios - testados)
    return {"cobertura_pct": pct, "faltantes": faltam}

MATRIZ = {"cenarios": [
    {"teste": "sessao_ativa_dentro_do_limite"},
    {"teste": "sessao_expira_no_limite_exato"},
    {"teste": "requisicao_longa_renova_sessao"},
    {"teste": "sessao_com_skew_de_relogio"},
    {"teste": "sessoes_concorrentes_independentes"},
]}

# O agente testou so o caminho feliz
testados = {"sessao_ativa_dentro_do_limite"}
resultado = cobertura_refutacao(MATRIZ, testados)
print(f"Cobertura: {resultado['cobertura_pct']}%")
print(f"Faltantes: {resultado['faltantes']}")

```

A cobertura de 20% — um cenário de cinco — reprova o artefato com evidência objetiva. A régua da cobertura transforma a refutação de opinião em métrica: o radar mostra o que falta, e o produtor sabe exatamente o que preencher.

O Modelo de Verificação por Camadas com Evidência

Cada camada da verificação produz evidência — e a evidência precisa ser rastreável até o artefato. O modelo abaixo registra o parecer de cada camada com o hash do artefato verificado:

```
import hashlib

class VerificacaoPorCamadas:
    def __init__(self):
        self.pareceres = []

    def verificar(self, camada, artefato, conteudo, aprovado, observacoes):
        hash_artefato = hashlib.sha256(conteudo.encode()).hexdigest()[:12]
        self.pareceres.append({'camada': camada, 'artefato': artefato,
                              'hash': hash_artefato,
                              'aprovado': aprovado, 'observacoes':
                              observacoes})
        return self.pareceres[-1]

    def aprovacoes_por_artefato(self, artefato):
        return [p for p in self.pareceres if p['artefato'] == artefato]

v = VerificacaoPorCamadas()
v.verificar('sintaxe', 'modulo_sessao.py', 'def expira(): pass', True, '')
v.verificar('logica', 'modulo_sessao.py', 'def expira(): pass', False,
'tempo de expiracao fixo')
print(v.aprovacoes_por_artefato('modulo_sessao.py'))
```

O hash liga cada parecer ao conteúdo exato verificado — se o arquivo mudar depois da aprovação, o hash não bate mais e a aprovação perde validade. Isso mata a aprovação fantasma: “foi aprovado no review” já não é argumento se o código que entrou no deploy não é o código que o parecer viu. A verificação por camadas com evidência hashada é a base de confiança de toda a esteira de refutação.

O Padrão de Evidência em Cada Camada

Vamos acompanhar um caso completo de verificação adversarial. O cenário: uma feature de sessão onde o agente implementou a expiração — e o revisor adversarial encontrou o defeito que o teste do agente não cobria.

1. **O agente implementa** a expiração e roda seus testes: o cenário feliz passa.

2. **O revisor adversarial pergunta:** e se a sessão estiver ativa durante uma requisição longa? E se houver skew de relógio? E se houver duas sessões concorrentes?
3. **O revisor escreve os testes de borda:** `requisicao_longa_renova_sessao`, `sessao_com_skew_de_relogio`, `sessoes_concorrentes_independentes`.
4. **Os testes de borda falham** — o defeito existe, com evidência.
5. **O agente corrige** contra os testes de borda; a suíte completa fica verde.

O código abaixo simula a descoberta do defeito:

```
class Sessao:
    def __init__(self, criada_em: int, duracao_seg: int = 1800) -> None:
        self.criada_em = criada_em
        self.duracao_seg = duracao_seg

    def expirada(self, agora: int, ultima_atividade: int) -> bool:
        # BUG: usa criada_em em vez de ultima_atividade
        return agora - self.criada_em > self.duracao_seg

sessao = Sessao(criada_em=100)
# usuario ativo o tempo todo: ultima_atividade recente, mas expiracao
baseada em criada_em
print("Expirada? ", sessao.expirada(agora=2000, ultima_atividade=1990))
```

O defeito é clássico: a expiração usa a criação em vez da última atividade — o cenário feliz passa, o cenário de requisição longa quebra. O revisor adversarial encontrou com evidência o que a auto-validação teria perdido.

O Padrão de Evidência em Cada Camada

A evidência tem formato por camada — e o parecer só é aceito quando a evidência casa com o formato esperado. O quadro abaixo é a régua de evidência:

Camada	Evidência aceita	Formato	Exemplo
Máquina	Saída de comando	Texto com exit code	pytest: 47 passed (exit 0)
Adversarial	Parecer estruturado	JSON com refutações	[{cenario, resultado, evidencia}]
Humana	Decisão registrada	Entrada de log	DEC-042: merge autorizado

A régua de evidência responde à pergunta que mata pareceres vagos: “o que conta como prova nesta camada?”. O revisor que entrega “revisei e está ok” sem o formato esperado é devolvido — evidência é formato, não intenção.

O Modelo de Rastreabilidade Evidência-Requisito

A refutação só tem valor se a evidência aponta para o requisito exato que ela defende. O modelo abaixo cria a ligação evidência-req e detecta requisitos sem nenhuma evidência de refutação — o furo no radar:

```
class RastreabilidadeEvidencia:
    def __init__(self):
        self.ligacoes = {}

    def ligar(self, evidencia, requisito, status):
        self.ligacoes.setdefault(requisito, []).append({'evidencia':
evidencia, 'status': status})

    def furos(self):
        return {req: evs for req, evs in self.ligacoes.items() if
all(e['status'] != 'aprovado' for e in evs)}

    def cobertura(self):
        aprovados = sum(1 for evs in self.ligacoes.values() if
any(e['status'] == 'aprovado' for e in evs))
        return aprovados / len(self.ligacoes) if self.ligacoes else 0

r = RastreabilidadeEvidencia()
r.ligar('teste_de_expiração_de_sessão.py', 'R7', 'aprovado')
r.ligar('revista_de_codigo_14.txt', 'R7', 'aprovado')
r.ligar('prova_manual_de_UX.txt', 'R12', 'pendente')
print(r.cobertura())
print(r.furos())
```

O requisito R12 com apenas uma evidência pendente aparece como furo no radar — o parecer do revisor humano ainda não chegou, e a entrega não pode avançar. A cobertura geral é a métrica que o comandante acompanha no painel: abaixo de 100%, há requisitos voando sem radar, e isso é inaceitável para aterrissagem.

A Fila de Refutação com Prioridade por Risco

Cada ciclo de verificação produz um relatório de refutação — o artefato que a esteira consome para decidir o avanço. O formato abaixo é o padrão:


```
{
  "ciclo_verificacao": "V-2026-031",
  "artefato": "feature/cupons-v2",
  "camadas": {
    "maquina": {"exit_code": 0, "saida": "47 passed, 0 failed"},
    "adversarial": {
      "refutacoes": [
        {"cenario": "cupom acumulativo", "resultado": "nao_refutado"},
      ],
      "evidencia": "teste canonico presente"
    },
    "parecer": "aprovado_com_ressalva"
  },
  "humana": {"decisao": "autorizar_merge", "decisor": "lead-plataforma"}
}
```

O relatório de refutação é a caixa-preta da Fase 5: três camadas, cada uma com sua evidência, tudo registrado em um único artefato consultável — o radar documentado, não o radar adivinhado.

A Fila de Refutação com Prioridade por Risco

Nem toda refutação tem o mesmo peso — e a fila de verificação deve priorizar por risco. O código abaixo estende a FilaVerificacao com prioridade: artefatos de alto risco (mudança de schema, regras de negócio) são refutados primeiro, com revisor humano obrigatório.

```
from dataclasses import dataclass
from typing import List

@dataclass
class ItemRefutacao:
    id: str
    risco: str # baixo | medio | alto
    produtor: str
    artefato: str
    revisor: str = ""

    def prioridade(self) -> int:
        return {"baixo": 1, "medio": 2, "alto": 3}[self.risco]

def fila_por_risco(itens: List[ItemRefutacao]) -> List[ItemRefutacao]:
    return sorted(itens, key=lambda i: -i.prioridade())
```

```

ITENS = [
    ItemRefutacao("I1", "baixo", "agente", "documentacao_atualizada.md"),
    ItemRefutacao("I2", "alto", "agente", "migracao_schema.sql"),
    ItemRefutacao("I3", "medio", "agente", "endpoint_novo.py"),
]

for item in fila_por_risco(ITENS):
    print(f"prioridade={item.prioridade()} {item.artefato} (risco {item.risco})")

```

A priorização por risco é o reflexo da torre: a aeronave em emergência aterrissa antes da que está em cruzeiro. O artefato de alto risco é refutado primeiro, com o revisor mais experiente.

O Modelo de Calibragem do Revisor Automatizado

Um revisor automatizado tende a ser permissivo ou severo demais com o tempo. O modelo abaixo calibra o revisor comparando seus pareceres com os do revisor humano numa amostra — o desvio vira fator de correção:

```

def calibrar_revisor(pareceres_auto, pareceres_humano):
    assert len(pareceres_auto) == len(pareceres_humano)
    falsos_positivos = sum(1 for a, h in zip(pareceres_auto,
pareceres_humano) if a == 'rejeitar' and h == 'aprovar')
    falsos_negativos = sum(1 for a, h in zip(pareceres_auto,
pareceres_humano) if a == 'aprovar' and h == 'rejeitar')
    total = len(pareceres_auto)
    return {
        'falsos_positivos': falsos_positivos,
        'falsos_negativos': falsos_negativos,
        'precisao': round((total - falsos_positivos - falsos_negativos) /
total, 2),
        'acao': 'ajustar_limiar' if falsos_positivos > falsos_negativos
else 'relaxar_limiar' if falsos_negativos > falsos_positivos else 'manter',
    }

print(calibrar_revisor(['aprovar', 'rejeitar', 'aprovar', 'rejeitar'],
['aprovar', 'aprovar', 'aprovar', 'rejeitar']))

```

A calibragem transforma o erro do revisor em dado: se o revisor automatizado rejeita mais que o humano (falsos positivos), o limiar está severo demais e a fila de trabalho incha; se aprova mais que o humano (falsos negativos), o radar está frouxo e defeitos passam. A amostragem periódica com humanos é o que impede o revisor de derivar silenciosamente — o radar vigia o radar.

A Amostragem de Pareceres como Auditoria

O radar do radar — a auditoria dos pareceres — precisa de método. A amostragem estatística é a prática: de cada lote de pareceres aprovados, uma amostra é reavaliada contra o que aconteceu depois em produção.

```
import random

def amostrar_pareceres(pareceres: list, taxa: float = 0.1) -> list:
    """Seleciona amostra aleatoria de pareceres para auditoria manual."""
    random.seed(42)
    n = max(1, round(len(pareceres) * taxa))
    return random.sample(pareceres, k=n)

PARECERES = [
    {"id": i, "camada": "adversarial", "aprovado": True, "artefato":
f"feature-{i}"},
    for i in range(50)
]

amostra = amostrar_pareceres(PARECERES)
print(f"Auditar {len(amostra)} de {len(PARECERES)} pareceres:")
for p in amostra:
    print(f" - parecer {p['id']} ({p['artefato']})")
```

A amostragem transforma a auditoria de pareceres em rotina barata: 10% dos pareceres reavaliados mantêm o radar honesto sem custar o ciclo inteiro.

O Modelo de Cobertura de Refutação por Risco

O radar não protege tudo igualmente — deve concentrar refutação onde o risco é maior. O modelo abaixo aloca o esforço de refutação proporcionalmente ao risco de cada artefato:

```
def alocar_refutacao(artefatos):
    total_risco = sum(a['risco'] for a in artefatos)
    for a in artefatos:
        a['peso_refutacao'] = round(a['risco'] / total_risco, 2)
    return sorted(artefatos, key=lambda x: x['peso_refutacao'],
reverse=True)

artefatos = [
    {'artefato': 'modulo_pagamentos.py', 'risco': 9},
    {'artefato': 'modulo_relatorio.py', 'risco': 3},
    {'artefato': 'modulo_avatar.py', 'risco': 1},
```

```

]
print(alocar_refutacao(artefatos))

```

A alocação por risco responde à pergunta incômoda do radar: refutação é cara, e o esforço deve seguir o dinheiro. O módulo de pagamentos (risco 9) recebe nove vezes mais esforço de refutação que o módulo de relatório (risco 3) e vinte e sete vezes mais que o avatar (risco 1). Refutar tudo com o mesmo peso é teoricamente bonito e operacionalmente irresponsável — o radar que vigia tudo por igual acaba vigiando mal o que importa.

O Radar do Radar

Se a verificação é o radar, quem verifica a verificação? A resposta é a auditoria de pareceres: uma amostra periódica das refutações e aprovações, comparada com o que aconteceu depois em produção. Quando um parecer aprovou um artefato que depois falhou em produção, o radar falhou — e a falha vira insumo do debriefing (Capítulo 8), não vergonha. Essa segunda camada de observação é o que impede o sistema de verificação de virar ritual: o radar que nunca falha é o radar que ninguém audita.

O Orçamento de Verificação por Camada

A verificação também tem orçamento — cada camada consome tokens, e o Comandante aloca o combustível do radar com a mesma disciplina das fases de produção:

Camada	Função	Custo relativo	Alocação típica
Máquina	typecheck/ lint/testes	Baixo	20% do orçamento de verificação
Adversarial	refutação com evidência	Médio	50%
Humana	arbitragem de exceção	Alto	30% (e não escala)

A alocação reflete a regra de ouro do radar: a maior parte do orçamento vai para a camada que filtra volume (adversarial), e a camada humana — cara e insubstituível — fica para a exceção, não para o volume.

O Modelo de Gate de Refutação por Critério

O gate de verificação precisa de critérios mensuráveis — não de parecer subjetivo. O modelo abaixo valida um artefato contra critérios explícitos e emite o parecer:

```
CRITERIOS_DE_GATE = [
    {'id': 'C1', 'descricao': 'testes rodando no CI'},
    {'id': 'C2', 'descricao': 'sem pendencia de revisao'},
    {'id': 'C3', 'descricao': 'diagrama atualizado'},
    {'id': 'C4', 'descricao': 'referencias rastreaveis'},
]

def avaliar_gate(resultados):
    cumpridos = [c for c in CRITERIOS_DE_GATE if resultados.get(c['id'])]
    faltantes = [c['id'] for c in CRITERIOS_DE_GATE if not
resultados.get(c['id'])]
    return {'aprovado': len(cumpridos) == len(CRITERIOS_DE_GATE),
'cumpridos': [c['id'] for c in cumpridos], 'faltantes': faltantes}

print(avaliar_gate({'C1': True, 'C2': True, 'C3': False, 'C4': True}))
```

O parecer deixa de ser “o revisor achou bom” e vira “quatro critérios, três cumpridos, falta o diagrama”. O gate com critérios explícitos também ensina o que a organização valoriza — cada critério listado é um compromisso visível. Quando o C3 (diagrama) falha sempre, a conversa não é sobre rigor do revisor, é sobre por que os diagramas não acompanham o código.

A Curva de Custo da Refutação

Há uma métrica econômica que justifica todo o capítulo: a curva de custo da refutação. Corrigir um defeito na Fase 5 custa, em média, uma fração do que custa corrigir o mesmo defeito na Fase 7 em produção. No SDLC AI-first, essa fração é medida em tokens — e é a melhor taxa de retorno de investimento do ciclo inteiro. Cada token gasto em refutação antecipada economiza dezenas de tokens em retrabalho tardio, sem contar o custo invisível do incidente: clientes, reputação e contexto de emergência.

O Painel de Saúde do Radar

O radar precisa de um painel que mostre sua própria saúde — número de refutações por semana, tempo médio de refutação, falsos positivos detectados na calibragem, requisitos sem evidência. O painel responde à pergunta que ninguém faz: o radar está funcionando ou está só rodando? Refutação que não muda decisão é teatro; o painel mostra quando a verificação virou burocracia — refutações em alta sem nenhum bloqueio, ou pior, sem nenhuma aprovação com ressalva. O painel de saúde é o que separa a esteira que verifica da esteira que finge verificar.

Passos para Implantar o Radar

1. **Comece pela máquina:** typecheck, lint e testes rodando em toda mudança — sem exceção.
2. **Adicione o revisor adversarial** com postura explícita de refutação, com evidência estruturada em cada parecer.
3. **Defina a porta de evidência:** merge bloqueado se qualquer camada não registrar saída verde.
4. **Reserve o humano para a exceção:** a decisão de merge em mudanças de contrato e impacto em produção.
5. **Meça a eficácia do radar:** quantas refutações cada camada pegou antes de produção.

Aplica

Cena real, em segunda pessoa. Sua fintech cresceu e o time de plataforma delegou features inteiras a agentes. O processo atual: o agente escreve, roda os testes locais, abre o PR, e o lead — você — aprova depois de uma leitura de 10 minutos. Nos últimos dois meses, três incidentes em produção rastream a causa até “o caso que ninguém testou”: uma fatura duplicada, um race condition no estorno e uma sessão que não expirava.

O erro não é o agente escrever código. O erro é o radar mudo. As três camadas existiam de nome — typecheck, review, QA — mas nenhuma tinha postura adversarial nem exigia evidência. O agente dizia “testei”, e a palavra bastava. A sessão que não expirava era exatamente o caso de borda que o teste do agente não cobria — porque o agente testou o que imaginou, não o que o radar exigiria.

O diagnóstico, ligado à teoria: verificação sem postura adversarial é validação do produtor. A correção prática:

1. **Camada 1 primeiro:** CI obrigatório com typecheck, lint e testes em toda mudança — o PR do agente não existe sem o verde.
2. **Revisor adversarial agêntico** em todo PR do agente, com a régua de refutação e parecer com evidência estruturada.
3. **Porta de evidência:** merge automatizado só quando as três camadas registrarem saída verde em arquivo.
4. **Humano nas exceções:** você não lê todo PR — lê os pareceres e arbitra os casos em que máquina e revisor discordam.

Armadilhas comuns: confundir cobertura de testes com cobertura de borda (o teste que só cobre o caminho feliz é um radar que enxerga só a pista principal); aceitar parecer sem evidência (“revisei e ok” sem o output); e transformar a revisão em ritual de aprovação — se o revisor nunca reprova nada, ele não está revisando.

Conclusão

Você acionou o radar. Três marcos: primeiro, as três camadas de verificação — máquina, adversarial e humana — cada uma com função distinta e postura própria; segundo, a evidência antes de afirmação como princípio inegociável, materializado em pareceres estruturados e porta de evidência no CI; terceiro, a refutação como vocabulário profissional — o revisor caça o defeito com prova, não valida o acerto com elogio.

Como desafio, implemente a porta de evidência no seu repositório: nenhum merge sem a saída registrada das três camadas em arquivo.

No próximo capítulo, você autoriza a aterrissagem: release e observabilidade — como entregar com segurança e monitorar o comportamento do próprio agente em produção.

Por que este capítulo importa

Se você chegou até aqui, já percebeu que o radar: verificação adversarial e evidência. Este capítulo — *Capítulo 6: O Radar: Verificação Adversarial e Evidência* — é um convite para parar de usar IA como ferramenta de autocomplete e começar a tratá-la como parte estrutural do seu processo de desenvolvimento. A diferença entre as duas posturas é exatamente o que separa quem apenas acelera o que já fazia de quem repensa o que é possível.

Vamos explorar isso com exemplos práticos, código real e um passo a passo que você pode aplicar ainda hoje, sem esperar por infraestrutura nova ou aprovação de comitê. A ideia é simples: cada seção termina com algo que você pode executar em menos de uma hora.

Conceitos-chave deste capítulo

- **O contrato antes da execução:** antes de deixar qualquer agente trabalhar, defina o que significa “feito” em termos verificáveis.
- **Evidência antes de afirmação:** o que não pode ser verificado não pode ser delegado com segurança.
- **Aprendizado contínuo:** cada entrega, boa ou ruim, é matéria-prima para o próximo ciclo.

Esses três conceitos aparecem, de formas diferentes, em todas as seções a seguir. Mantê-los em mente enquanto você lê vai transformar exemplos isolados em um padrão que você reconhece na sua própria rotina.

Checklist para aplicar hoje

1. Escolha uma tarefa pequena e bem definida do seu backlog.
2. Escreva o critério de aceite em uma frase verificável.
3. Delegue a execução a um agente, mantendo a verificação em suas mãos.
4. Registre o que funcionou e o que não funcionou.
5. Transforme o aprendizado em um procedimento reutilizável.

Se você fizer apenas o primeiro item, já estará à frente da maioria das equipes — que continua discutindo IA em reuniões sem nunca definir o que quer que ela faça.

Perguntas que você deve se fazer

1. Qual fase do meu processo consome mais tempo hoje — e por quê?
2. O que eu delegaria a um agente amanhã se tivesse certeza de que o resultado seria verificado?
3. Qual informação eu poderia registrar hoje que tornaria a próxima iteração mais barata?
4. Quem no meu time revisa o trabalho de quem — e com qual critério?
5. O que eu faria se o custo de cada tentativa caísse para quase zero?

Essas perguntas não têm resposta certa, mas têm uma propriedade em comum: elas forçam você a sair da conversa abstrata sobre IA e entrar no terreno do seu processo real. E é exatamente nesse terreno que o SDLC AI-first produz resultado.

Glossário rápido

- **Agente:** programa que usa um modelo de linguagem para planejar e executar tarefas com acesso a ferramentas.
- **Harness:** a camada que conecta o agente ao ambiente — arquivos, comandos, testes e regras.
- **Spec executável:** especificação cujos critérios podem ser verificados por máquina.
- **Verificação adversarial:** camada que refuta o trabalho produzido, em vez de apenas confirmá-lo.
- **Contexto:** a janela de informação que o modelo enxerga a cada passo — o recurso mais caro do ciclo.

Dominar esses cinco termos é suficiente para acompanhar qualquer discussão séria sobre desenvolvimento orientado a agentes.

O erro mais comum nesta fase

A maioria das equipes comete o mesmo erro: adota a ferramenta e mantém o processo. O agente entra no fluxo como um autocomplete sofisticado, e todo o potencial de transformação se perde em pequenas conveniências. O antídoto é simples e desconfortável: mude o processo primeiro, depois traga a ferramenta. Defina o contrato, o critério de aceite e a verificação antes de permitir que o agente produza em escala. É contra intuitivo, mas é o que separa as equipes que capturam valor das que apenas geram volume.

Um exemplo concreto para fixar

Imagine uma pequena feature de faturamento. No fluxo tradicional, um desenvolvedor recebe a tarefa, interpreta a intenção, escreve o código e um revisor confia na leitura. No fluxo orientado a agentes, a mesma feature começa com uma frase verificável: “o valor total deve considerar o desconto aplicado antes dos impostos”. O agente implementa, os testes verificam a regra, e o humano revisa a evidência — não o código linha a linha, mas o comportamento observado. Perceba o deslocamento: o humano deixa de ler tudo para auditar o essencial, e o agente deixa de adivinhar para executar contra um critério. É essa troca que o restante do livro explora em profundidade.

A rotina de quem já opera assim

Uma semana de trabalho em uma equipe que já adotou o ciclo orientado a agentes não parece uma revolução — parece um fluxo calmo e bem definido. Na segunda-feira, a especificação da semana é revisada em uma reunião curta: cada critério de aceite é lido em voz alta e qualquer ambiguidade é resolvida antes de tocar em código. Na terça, os agentes executam as tarefas em isolamento, enquanto os humanos revisam a arquitetura e os contratos. Na quarta, a verificação roda: testes, revisão adversarial e a decisão de merge apoiada em evidência. Na quinta, o que passou vai para produção em canário, com observabilidade ligada. Na sexta, o debriefing transforma os incidentes da semana em lições e skills. Nenhum dia é heroico; todos os dias são previsíveis. E é exatamente essa previsibilidade — não a velocidade máxima — que define a alta performance.

O que não fazer: os anti-padrões mais comuns

Se você quer destruir o valor do desenvolvimento orientado a agentes, aqui estão as receitas mais eficientes. Primeiro, o prompt-and-pray: gere o código, olhe por cima, e peça desculpas quando quebrar. Funciona em demos, falha em produção. Segundo, a spec decorativa: escreva documentos longos que ninguém verifica e que o agente não consegue executar — o pior dos dois mundos. Terceiro, a auto-verificação: deixe que quem escreveu valide o próprio trabalho, sem revisor independente; é a forma mais rápida de transformar confiança em acidente. Quarto, a delegação sem observabilidade: conceda autonomia sem instrumentar o comportamento. Todos esses padrões têm uma origem comum — a pressa em capturar o ganho sem construir o controle. E todos têm o mesmo antídoto: contrato antes de execução, evidência antes de afirmação, e revisão independente em toda entrega.

Como medir o progresso na prática

Uma dúvida legítima é: como saber se a adoção está dando certo? Métricas tradicionais de velocidade podem enganar — um time pode entregar mais rápido e acumular dívida técnica invisível. O indicador mais confiável no ciclo orientado a agentes é a estabilidade: quantos incidentes em produção, quanto tempo de retrabalho, quantas correções de emergência. Um segundo indicador é o custo de contexto: quantos tokens cada fase consome, e onde o desperdício se concentra. Um terceiro é a taxa de aceite na primeira verificação: se os agentes precisam de muitas rodadas de refutação, o contrato está fraco — o problema não é o agente, é a spec. Com esses três números na mesa, a conversa de progresso deixa de ser anedótica e vira análise de processo.

O papel do líder neste capítulo

Nada do que este capítulo descreve acontece por acaso — alguém precisa criar as condições para que o processo exista. Esse alguém é o líder técnico, o líder de equipe ou o arquiteto que decidiu tratar o ciclo orientado a agentes como uma mudança de processo, não como a instalação de uma ferramenta. O trabalho do líder aqui tem quatro frentes. A primeira é a modelagem do contrato: garantir que cada fase tem entrada, saída e critério definidos. A segunda é a calibragem da confiança: decidir o que pode ser delegado e o que exige decisão humana, e documentar essa decisão. A terceira é a defesa do tempo de verificação: em uma cultura que celebra velocidade, o líder precisa defender o orçamento de revisão como quem defende o seguro do prédio. A quarta

é o exemplo: o líder que pede evidência antes de afirmar, em toda reunião, ensina mais do que qualquer documento de processo.

Perguntas frequentes honestas

P: Isso não vai tirar o emprego dos desenvolvedores? R: A história do ciclo de vida nunca foi sobre menos trabalho humano, mas sobre trabalho mais valioso. O que muda é a natureza da tarefa: escrever código repetitivo deixa de ser o centro, e a especificação, a verificação e o desenho de contratos ocupam o lugar. P: Precisamos de um time de especialistas em IA para começar? R: Não. Precisa-se de disciplina de processo e de vontade de medir. As ferramentas evoluem rápido; o processo é o que permanece. P: E se o agente produzir código que ninguém entende? R: Essa é a pergunta certa — e a resposta é a verificação: se o código passa nos testes, na revisão adversarial e na observabilidade em produção, o fato de ter sido escrito por um agente é irrelevante. O critério não é a origem, é a evidência. P: Quanto tempo leva para ver resultado? R: Na primeira semana você já vê o efeito de escrever critérios de aceite verificáveis, independentemente de agentes. Os ganhos estruturais aparecem em um a dois ciclos.

Um convite para a prática deliberada

Conhecimento sem prática é entretenimento disfarçado de aprendizado. Este capítulo termina com um convite para a prática deliberada: escolha um artefato real do seu trabalho — uma spec, um teste, um release — e aplique deliberadamente um dos conceitos aqui descritos. Anote o antes e o depois. Repita por quatro semanas. No fim do mês, compare: o processo está mais previsível? O custo de contexto caiu? A estabilidade melhorou? Esse experimento pessoal, pequeno e mensurável, vale mais do que qualquer curso. É assim que o ciclo orientado a agentes deixa de ser um conceito que você explica para outras pessoas e se torna uma capacidade que você demonstra.

Síntese para levar com você

Se você guardar apenas uma ideia deste capítulo, que seja esta: no ciclo orientado a agentes, o contrato precede a execução, a evidência precede a afirmação e a revisão independente precede a entrega. Tudo o mais — as ferramentas, os modelos, os fluxos — muda rápido e pode ser aprendido conforme a necessidade. O que não muda é a disciplina: sem ela, a IA é um gerador

de volume; com ela, é um multiplicador de capacidade. O resto do livro é a expansão dessa disciplina em cada fase do ciclo de vida.

De onde veio a necessidade desta mudança

Vale a pena entender por que este capítulo existe — e por que ele não foi escrito dez anos atrás. A resposta está na economia do desenvolvimento de software. Durante décadas, o custo dominante de produzir software foi o trabalho humano: escrever, revisar, corrigir. Todo o ciclo de vida clássico foi desenhado em torno dessa escassez — processos, papéis e artefatos existem para coordenar pessoas e evitar retrabalho caro. O que mudou nos últimos anos foi a emergência de modelos capazes de gerar, revisar e executar código com custo marginal próximo de zero. De repente, a escassez dominante não é mais a mão de obra: é a capacidade de especificar, orquestrar e verificar. Esse deslocamento — de horas-homem para tokens e contexto — é a raiz de tudo o que este capítulo descreve. Quem entende essa mudança de economia entende por que o processo precisa mudar junto com a ferramenta.

Conversando com quem resiste

Em toda equipe há quem resista à mudança — e a resistência quase nunca é preguiça, é uma pergunta legítima sem resposta. As objeções mais comuns são três. “Já tentamos automação e quebrou”: a resposta é que a automação anterior quebrou porque o processo não tinha contrato nem verificação; é exatamente isso que o novo ciclo constrói antes de automatizar. “IA gera código que ninguém entende”: a resposta é que o critério de entendimento mudou — o que importa não é a origem do código, mas se ele passa na verificação; e a revisão de contrato e arquitetura continua humana. “Isso é modismo”: a resposta mais honesta é que pode ser, mas o processo que este capítulo descreve — especificar, delegar, verificar, aprender — melhora o ciclo com ou sem IA. A disciplina é o investimento à prova de modismo.

O dia a dia no detalhe

Para tornar concreto o que este capítulo descreve, vale percorrer o dia a dia de uma tarefa típica, passo a passo. A manhã começa com a revisão da intenção: o produto explica o que quer, o time traduz em requisitos com critérios verificáveis e ninguém toca em código antes de a spec estar aprovada. Na sequência, o trabalho é despachado: cada tarefa vai para um contexto isolado, com seu contrato anexado. A execução produz artefatos — código, testes, diagramas — e cada

artefato carrega a evidência de como foi produzido. A tarde é de verificação: testes automáticos, revisão adversarial e a leitura humana do que é crítico. O que passa, segue; o que não passa, volta com o parecer anexado — sem discussão de opinião, porque o critério já estava escrito. O fim do dia é de registro: o que foi aprendido, o que custou em contexto, o que deve mudar no processo. Esse fluxo parece simples, mas cada passo exige disciplina — e é exatamente a simplicidade do ritmo que o torna sustentável.

O custo invisível que decide tudo

Há um recurso que atravessa todos os exemplos deste capítulo e que raramente aparece nas discussões: o contexto. Cada interação com um modelo de linguagem consome uma janela de informação — e essa janela é limitada e cara. Uma spec mal escrita gasta contexto em ciclos de correção. Um log inteiro no contexto gasta contexto que poderia servir à verificação. Uma busca redundante gasta contexto sem produzir informação. Quem ignora esse custo descobre, cedo ou tarde, que a automação ficou mais cara que o trabalho manual que pretendia substituir. Por isso a disciplina de contexto não é um detalhe de economia — é uma decisão de arquitetura do ciclo. Medir o consumo por fase, comprimir o que é ruído e injetar apenas o necessário são práticas que determinam se o SDLC AI-first se sustenta em escala. Este capítulo toca nesse tema; os capítulos finais do livro o desdobram em técnica.

Uma visão de longo prazo

A adoção do ciclo orientado a agentes não é um projeto com data de fim — é uma trajetória que se desenrola ao longo de anos, e vale a pena olhar para a frente. No primeiro trimestre, o foco é o contrato: as equipes aprendem a especificar e a verificar, e os ganhos vêm da clareza, não da automação. No segundo trimestre, a delegação supervisionada entra em produção em áreas de baixo risco, e as métricas começam a mostrar onde o ciclo ganha e onde perde. No segundo ano, o ciclo adversarial se consolida: verificação automática, observabilidade do comportamento agêntico e aprendizado organizacional rodando como rotina. No terceiro ano, a organização opera em um nível de maturidade em que a IA é parte estrutural do processo, e a pergunta deixa de ser “como adotar” e passa a ser “como evoluir”. Quem inicia com o processo antes da ferramenta chega a esse destino com estabilidade; quem inverte a ordem, chega com dívida. A escolha é sua.

Próximos Passos

Você acabou de percorrer um dos capítulos centrais do SDLC AI-first. Se este conteúdo fez sentido para você, o próximo passo natural é continuar a jornada pelo livro completo, que aprofunda cada fase do ciclo: especificação executável, design de domínio, harness e agentes, verificação adversarial, entrega segura, aprendizado contínuo, economia de tokens e governança de maturidade.

Enquanto isso, aqui vão três ações concretas:

1. **Pratique em um projeto pequeno.** Nada substitui experimentar com as próprias mãos — escolha um repositório pessoal e aplique um dos conceitos deste capítulo.
2. **Formalize seu contrato.** Escreva o critério de aceite de uma tarefa real antes de delegá-la. É um exercício de cinco minutos que muda completamente a qualidade do resultado.
3. **Mensure seu processo.** Registre quanto tempo e quanto contexto cada fase consome. O que não é medido não pode ser melhorado.

O céu do software agora tem torres de controle. O próximo voo é seu.

Boa leitura e bons voos.

— Heverton Eduardo Peres.

